40. Shell Text Mastery Drill Week

40. Shell Text Mastery Drill Week

Phase: Phase 3 — Text, pipes, and discovery
Estimated time: 2–3 days
Slug: `40-shell-text-mastery`

Learning objectives

- Combine grep/sed/awk/sort/uniq fluently
- Write one-liners you can explain line-by-line
- Profile slow pipelines
- Prefer clarity over golf

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Linux administrators spend a surprising share of their time turning text into answers: which IP hits us most, how many 5xx errors in the last hour, which users have no shell, which configuration value differs between servers. Mastery of a small toolbox (`grep`, `cut`, `sort`, `uniq`, `awk`, `sed`, `xargs`, `tr`) lets you answer such questions in one line, without writing a program. This module consolidates earlier text lessons into reusable **recipes**, including the "top N" pattern that powers the exercise.

Core concepts

Tool	Best at
`grep`	Select lines by pattern
`cut`	Take fixed delimiter fields or char ranges
`awk`	Fields, arithmetic, conditions, simple reports
`sed`	Stream edits: substitute, delete, print ranges
`sort`	Order lines (`-n` numeric, `-r` reverse, `-k` key, `-u` unique)
`uniq`	Collapse **adjacent** duplicates; `-c` counts
`tr`	Translate or delete characters
`xargs`	Turn lines into command arguments
`paste`/`join`	Combine files side by side or by key

The universal pipeline pattern:

```
 source --> select --> extract --> normalize --> count/aggregate --> rank
 (cat)     (grep)     (awk/cut)   (tr/sed)      (sort | uniq -c)    (sort -nr |
head)
```

The frequency recipe, used in countless incidents:

```
 sort | uniq -c | sort -nr | head
```

`uniq` only merges adjacent lines, so you must `sort` before `uniq -c`. Then sort numerically descending to rank.

Worked examples

Sample data, `access.log`:

```
10.0.0.5 GET /index 200
10.0.0.7 GET /login 500
10.0.0.5 POST /login 200
10.0.0.5 GET /index 200
10.0.0.9 GET /admin 403
```

Top clients:

```
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head -n 3
```

```
      3 10.0.0.5
      1 10.0.0.9
      1 10.0.0.7
```

Most frequent token (the exercise function):

```
top_token() {
  sort | uniq -c | sort -nr | awk 'NR==1{print $2}'
}
printf 'a\nb\na\n' | top_token      # a
```

Count status codes and find errors:

```
awk '{c[$4]++} END{for(k in c) print k, c[k]}' access.log | sort
awk '$4>=500' access.log
grep -c ' 200$' access.log
```

Words from a text file, case-insensitive:

```
tr -cs 'A-Za-z' '\n' < README.md | tr 'A-Z' 'a-z' | sort | uniq -c | sort -nr |
head
```

System account recipes:

```
cut -d: -f1,7 /etc/passwd | grep -v 'nologin\|false'
awk -F: '$3>=1000 && $3<65000 {print $1}' /etc/passwd
```

Edit text safely:

```
sed -n '2,4p' access.log
sed 's/10\.0\.0\./net-A./' access.log | head -n 2
sed -i.bak 's/^PermitRootLogin yes/PermitRootLogin no/' sshd_config
```

Safe file handling with `xargs`:

```
find . -name '*.log' -print0 | xargs -0 grep -l ERROR
find . -name '*.tmp' -print0 | xargs -0 -r rm -v
```

Join two lists on a key:

```
sort -k1,1 users.txt > u.sorted; sort -k1,1 groups.txt > g.sorted
join u.sorted g.sorted
```

Diff and compare:

```
diff <(sort a.txt) <(sort b.txt)
comm -12 <(sort a.txt) <(sort b.txt)    # lines in both
```

Mini-lab

1. Create the sample log and reproduce the top-clients output.
2. Remove the first `sort` from the pipeline and observe wrong counts.
3. Write `top_token` and test with tied input like `a b b a`. Note which wins and why (the sort order decides ties).
4. From `/etc/passwd`, list users with login shells sorted by UID.
5. Extract all IPv4-like strings from a log with `grep -oE '[0-9]+(\.[0-9]+){3}'` and rank them.
6. Rewrite one of your pipelines as a function in a script with a usage line.

Decision guide / failure modes

Task	Use
Filter lines	`grep`
Split by delimiter, simple	`cut`
Conditions, sums, per-field logic	`awk`
Replace text	`sed`
Count distinct values	`sort | uniq -c`
Filenames with spaces	`-print0 | xargs -0`

Failure modes:

- `uniq` without `sort` gives misleading counts.
- Locale changes sort order; set `LC_ALL=C` for stable results in scripts.
- `cut` fails on variable-width whitespace; use `awk` instead.
- `sed -i` without a backup suffix destroys originals on a bad regex.

- `cat file | grep` is fine for exploration but needless in scripts.
- Parsing `ls` output; use globs or `find`.
- Whole-pipeline exit status is the last command's unless `pipefail` is set.

Common mistakes

- Unreadable golf one-liners in prod
- Loading entire huge files into arrays unnecessarily
- Locale surprises in sort

Exercises

Work in `exercises/40-shell-text-mastery/`. Read that folder's `README.md` fully before coding.

```
cd exercises/40-shell-text-mastery
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `top_token` in `exercise.sh`. It reads tokens from standard input, one per line, and prints only the most frequent token. Use the recipe from the lesson: sort, count with `uniq -c`, sort by count descending, then print the token column. Do not print the count. For ties, whichever token comes first after the numeric sort is acceptable. Test with piped input using `printf`, as `test.sh` does. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/40-shell-text-mastery/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. How top-N with sortluniq -c?
2. Why LC_ALL=C sometimes?
3. When rewrite as a script?

Further reading

- https://www.gnu.org/software/coreutils/manual/